ClubSphere: A Role-Based Web Platform for Centralizing College Club Administration and QR-Based Event Registration

Avantika Mishra, Avanshi Kothari, Akansha Sharma, Aditi Maheshwari
Department of Computer Science and Engineering
Arya College of Engineering and IT, Kukas, Jaipur, Rajasthan, India
Guided by Er. Ram Babu Buri, Department of CSE
PBL Team ID: PBL2627-CSE-A-024 · Academic Session 2026–27
Corresponding author: team PBL2627-CSE-A-024, Dept. of CSE, ACEIT, Jaipur

Abstract

Abstract—Student clubs in many engineering colleges still coordinate membership, event publicity and registrations through messaging groups, printed posters and spreadsheets. Those channels scatter information and leave access control to whoever administers each group. This paper presents ClubSphere, a server-rendered Java web application that centralizes the official club directory, event publishing, QR-based registration, club tasks, announcements and in-app notifications for four roles: administrator, club head, club member and student. The system is a layered modular web application built with Jakarta Servlets and JSP on Apache Tomcat 10.1, JDBC with HikariCP, and MySQL 8, with BCrypt password hashing and HTTP-session authentication. Its central design principle is that authorization is enforced in server-side code: an authentication filter guards role URL prefixes, club-head operations are scoped to an assigned club, duplicate event registrations are rejected by a unique constraint, and QR tokens map to a configured public registration URL. The implementation is accompanied by a reproducible schema and seed, a Docker Compose demonstration path, and automated unit tests for password hashing and QR PNG generation. No performance, usability or security-audit measurements are claimed; limitations are stated explicitly.

Index Terms*—Campus information systems, club management, event registration, QR codes, role-based access control, BCrypt, Jakarta Servlets, JSP, MySQL, Apache Tomcat.*

I. INTRODUCTION

Higher-education campuses depend on two kinds of extracurricular coordination: structured administration of official student societies (membership, faculty advisors, event calendars) and loosely structured publicity (posters, group chats, last-minute registration lists). Learning management systems address coursework [1], [2], and smart-campus literature discusses broader institutional integration [3]. In practice, club operations often remain outside those platforms. Each society keeps its own roster; students miss events; administrators cannot see users, clubs and registrations in one place.

ClubSphere was developed as a B.Tech. Project-Based Learning assignment (PBL2627-CSE-A-024) at Arya College of Engineering and IT to combine these functions in one application with one identity model. The first release is scoped to a single institution. This paper reports architecture, design decisions, verification evidence and the evaluation work that remains. It makes no claim of pedagogical impact or of performance at scale.

The main contributions are: (1) a four-role identity model (ADMIN, CLUB_HEAD, CLUB_MEMBER, STUDENT) enforced by a servlet filter and HTTP sessions rather than by hiding user-interface links; (2) a club-centric data model in which events, tasks, announcements and memberships reference official club records; (3) QR-based event registration in which each event carries a unique token rendered as a PNG whose payload is a configured public URL; and (4) a transparent verification record that distinguishes implemented behaviour from unmeasured quality attributes.

II. RELATED WORK

A. Learning Management and Smart Campus Platforms

Watson and Watson distinguish learning management systems from course management systems [1]. Moodle is a widely studied open-source platform grounded in community-oriented pedagogy [2]. Such systems provide mature assignment workflows, but club directories, QR event check-in and society-internal tasks usually sit outside that boundary. Min-Allah and Alrashed describe the smart campus as integration of services, infrastructure and data [3]. ClubSphere addresses a narrow, software-only slice of that vision: extracurricular administration.

B. Event Registration and QR Codes

QR codes are a practical way to carry a short URL into a physical venue [4]. Attendance and ticketing systems often encode opaque tokens rather than guessable identifiers so that scanning opens a specific registration or check-in resource. ClubSphere uses this idea for event signup: the token is stored with the event row and the image is generated on demand. The paper does not claim a new encoding algorithm; the contribution is binding the token to club-owned events and to session-authenticated registration.

C. Web Architecture and Access Control

Fowler's layered architecture, service layer and data-access patterns separate presentation, business rules and persistence [5]. For a four-member student team, a modular Java web application (servlet–service–DAO) is a defensible alternative to microservices [6]. Role-based access control assigns permissions to roles rather than to individuals [7], [8]. Pure

RBAC cannot express “a club head may edit only the club they head,” which depends on a relationship. Broken access control ranks highly in the OWASP Top 10, which recommends server-side enforcement and deny-by-default [9]. bcrypt makes password hashing cost tunable [10].

III. RESEARCH GAP AND OBJECTIVES

Existing work provides mature solutions to individual sub-problems: course management [1], [2], access-control models [7], [8], and QR encoding [4]. The project-level gap is engineering: integrating club catalogues, event capacity, QR signup, tasks and notifications under one identity model without losing consistent authorization or honesty about what has been verified.

O1: Provide one identity model for administrators, club heads, club members and students, with passwords stored as bcrypt hashes and sessions created only after verification.

O2: Maintain an official club directory and memberships that other modules can reference.

O3: Support event publishing, individual and group registration, and QR tokens that resolve to a registration URL.

O4: Provide role dashboards, club tasks, announcements and in-app notifications.

O5: Enforce authorization in server-side filters and servlets, consistent with OWASP guidance [9].

O6: Ship a reproducible schema, seed data, environment template and a documented demonstration path, with automated tests where they exist.

IV. PROPOSED SYSTEM ARCHITECTURE

ClubSphere follows a client–server architecture with a layered backend. A browser loads JSP pages and submits HTML forms over HTTP. Apache Tomcat 10.1 hosts the application. AuthenticationFilter intercepts requests, allows a public whitelist (landing page, login, club and event catalogues, QR images, static assets) and otherwise requires a session attribute currentUser. Role prefixes /admin, /club-head, /club-member and /student are authorized in the filter. Servlets call services; services call DAOs that execute parameterized SQL through HikariCP against MySQL 8 (database clubsphere_db). In production-style packaging, Maven builds a WAR; a multi-stage Docker image copies it as ROOT.war and binds Tomcat to the PORT environment variable.

A. Modules and Interfaces

Module 1—Authentication and user management (Avantika Mishra)—implements login and logout, bcrypt verification, session-fixation mitigation, the authentication filter, and the administrative user directory. Module 2—Club management (Avanshi Kothari)—implements public club search, administrative club records, assignment of a club head, and membership add/remove. Module 3—Events and QR registration (Akansha Sharma)—implements event create/edit/cancel, public event listing, individual and group registration, and ZXing PNG generation at /qr/{token}. Module 4—Dashboards, tasks and notifications (Aditi Maheshwari)—implements four role dashboards, task assignment and status updates, announcements, in-app notifications and report summaries.

Representative URLs include POST /login, GET /clubs, GET /events, GET/POST /event/register, GET /qr/{token}, /admin/*, /club-head/*, /club-member/*, /student/* and /notifications/*. Every write operation derives the acting user from the HTTP session; a caller-supplied role in a hidden field is not treated as authorization.

B. Data Model

MySQL stores ten application tables defined in database/schema.sql: users, clubs, club_members, events, registrations, event_participants, tasks, announcements, notifications and activity_logs. Users have a role enumeration and a status. Clubs optionally reference a head user. Events belong to a club, carry a unique qr_token and a capacity, and receive registrations. A unique constraint on (event_id, user_id) prevents duplicate signups. Group registrations store participant rows. Tasks and announcements reference a club. Notifications are per user. Character set is utf8mb4; the engine is InnoDB with foreign keys.

C. Event and QR Workflow

A club head creates an event; the service stores a unique token. GET /qr/{token} looks up the event, builds APP_BASE_URL/event/register?token=… and returns a PNG (error-correction level H). An unauthenticated scan redirects to login with a return URL. After login, RegistrationServlet accepts token or eventId, rejects duplicates, stores optional group participants, and increments registered_count. The public origin in the QR must match the deployed context path (named WAR versus Docker ROOT).

V. AUTHORIZATION AND CONSISTENCY DESIGN

A. Layered Authorization

Authorization combines RBAC [7], [8] with relationship checks. First, protected URLs require a session. Second, the filter restricts prefixes by role. Third, club-head servlets resolve the managed club from head_user_id and refuse to continue if none is assigned, so a head cannot operate on another society by guessing identifiers. Fourth, administrators cannot deactivate their own logged-in account. Fifth, the login redirect parameter is restricted to internal paths to avoid open redirects. UI menus may hide links, but that is a convenience, not the security boundary [9].

B. Consistency Mechanisms

Duplicate memberships and duplicate event registrations are rejected by unique constraints as well as by service checks. Event cancellation is an explicit status change. Passwords are stored only as bcrypt hashes [10]. SQL uses PreparedStatement. Configuration prefers environment variables over committed secrets; a placeholder replaces any real password in application.properties.

VI. SECURITY, PRIVACY, AND ETHICAL CONSIDERATIONS

Implemented controls include bcrypt password hashing [10]; session invalidation before a new authenticated session is

created; HttpOnly cookies; role filters; parameterized SQL; and generic user-facing errors. Administrators create accounts in this release; there is no public self-registration.

Known weaknesses remain. Session cookies are not marked Secure in the default web.xml (appropriate only behind HTTPS). There is no rate limiting, e-mail verification or password-reset workflow. QR codes encode a registration URL; they are not a cryptographic attendance proof. Demo seed accounts share a documented evaluation password and must not be used in production. The system processes names, e-mail addresses and student identifiers. Before deployment beyond a classroom demonstration, the institution should define retention policy, who may export registration lists, and how long activity logs are kept.

VII. EVALUATION METHODOLOGY

The available automated evidence consists of JUnit 5 tests for PasswordUtil and QRCodeUtil (five methods: hash/verify, salt diversity, null handling, PNG magic bytes and Base64 data URIs). These tests do not require MySQL. They show that hashing and QR rendering behave as specified in isolation. They do not demonstrate end-to-end registration against a live database. Manual demonstration uses seeded roles (administrator, club head, club member, student). Code coverage, load tests, usability studies and security audits have not been conducted. A future protocol aligned with ISO/IEC 25010 [11] should report sample sizes and environment details.

Functional suitability: exercise login, RBAC denials, club browse, event create, QR fetch and duplicate registration against a disposable MySQL instance, with traceability to the SRS.

Security: attempt /admin as a student, open-redirect payloads on login, and duplicate membership inserts; expected result is denial without leaking secrets.

Reliability: restart Tomcat with MySQL still running and confirm seeded accounts still authenticate.

Usability (future): task-based sessions measuring whether a new student can register for an event via QR, using the System Usability Scale [12].

VIII. IMPLEMENTATION AND DEPLOYMENT

The implemented stack is Java 17, Jakarta Servlet 6, JSP, JSTL, Bootstrap 5, HikariCP, MySQL Connector/J, jBCrypt, ZXing, Maven, Apache Tomcat 10.1, and JUnit 5. The team divided work by module and integrated on GitHub (Avantikamishra862/Arya_Clubsphere). Integration order is schema and seed, authentication, clubs, events/QR, then dashboards. A live demonstration is hosted at https://event-management-uylc.onrender.com/. Docker Compose starts MySQL and the application for local evaluation.

IX. LIMITATIONS

No performance, usability or security-audit measurements exist. Automated tests cover two utilities, not DAOs or the user interface. Accounts are created by administrators. Notifications are in-app only. QR codes encode registration URLs rather than a separate attendance credential. File uploads are not a first-class module. ClubSphere should be regarded as a working academic prototype whose institutional value remains to be established empirically.

X. CONCLUSION AND FUTURE WORK

This paper presented ClubSphere, a role-based web platform that integrates club administration, event publishing, QR registration, tasks and notifications for Arya College of Engineering and IT. Its main engineering contributions are a four-role session model enforced in a filter, a club-centric schema, and QR tokens bound to event rows. Future work should run the evaluation protocol of Section VII; add public self-registration with college-email checks; send e-mail reminders; introduce an attendance QR distinct from registration; expand automated tests; and enable Secure cookies under HTTPS.

REFERENCES

[1] W. R. Watson and S. L. Watson, “An argument for clarity: What are learning management systems, what are they not, and what should they become?” TechTrends, vol. 51, no. 2, pp. 28–34, 2007.

[2] M. Dougiamas and P. C. Taylor, “Moodle: Using learning communities to create an open source course management system,” in Proc. EdMedia, Honolulu, HI, USA, 2003, pp. 171–178.

[3] N. Min-Allah and S. Alrashed, “Smart campus—A sketch,” Sustainable Cities and Society, vol. 59, Art. no. 102231, 2020.

[4] ISO/IEC 18004:2015, Information technology—Automatic identification and data capture techniques—QR Code bar code symbology specification.

[5] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002.

[6] N. Dragoni et al., “Microservices: Yesterday, today, and tomorrow,” in Present and Ulterior Software Engineering. Cham, Switzerland: Springer, 2017, pp. 195–216.

[7] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” Computer, vol. 29, no. 2, pp. 38–47, 1996.

[8] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, “Proposed NIST standard for role-based access control,” ACM Trans. Inf. Syst. Secur., vol. 4, no. 3, pp. 224–274, 2001.

[9] OWASP Foundation, “OWASP Top 10:2025—A01 Broken Access Control,” 2025. [Online]. Available: https://owasp.org/Top10/

[10] N. Provos and D. Mazières, “A future-adaptable password scheme,” in Proc. USENIX Annu. Tech. Conf. (FREENIX Track), 1999.

[11] Systems and Software Engineering—SQuaRE—Product Quality Model, ISO/IEC 25010:2023, 2023.

[12] J. Brooke, “SUS: A ‘quick and dirty’ usability scale,” in Usability Evaluation in Industry, P. W. Jordan et al., Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.

[13] Jakarta Servlet 6.0 Specification, Eclipse Foundation.

[14] Apache Tomcat 10 Documentation. [Online]. Available: https://tomcat.apache.org/

[15] Oracle, MySQL 8.0 Reference Manual. [Online]. Available: https://dev.mysql.com/doc/

[16] ZXing (“Zebra Crossing”) barcode scanning library. [Online]. Available: https://github.com/zxing/zxing

[17] Arya College ClubSphere source repository. [Online]. Available: https://github.com/Avantikamishra862/Arya_Clubsphere

APPENDIX A — IMPLEMENTATION CHECKLIST

Derive the acting user only from the HTTP session; never trust a client-supplied role as authorization.

Enforce role prefixes in AuthenticationFilter and club-head scope via head_user_id.

Store passwords only as bcrypt hashes; keep production credentials out of Git.

Back “at most one registration per user per event” with a unique constraint.

Build QR payloads from APP_BASE_URL so tokens work on WAR, Docker and hosted origins.

Use parameterized SQL in every DAO.

Document demo accounts separately from production policy.

Run utility tests and a four-role manual walkthrough before any institutional demonstration.